

една цялостна техническа бележка за утре, максимално близка до примера ти, която да съдържа реалното състояние на проекта, плана, решенията, блок-схемите и точното място, от което продължаваме.

Бележка за продължаване — StudentManagement

1. Главна цел на проекта

Целта е постепенно да изградим **Student Management System**, като започнем от Students CRUD и надграждаме системата с:

1. 👤 **Students** — добавяне, преглед, редактиране и изтриване
2. 🔍 **Search** — търсене на студент
3. 📖 **Courses / Subjects** — предмети
4. 📝 **Grades** — оценки
5. 📊 **Average Grade** — автоматично изчисляване на среден успех
6. 📋 **Attendance** — присъствия
7. 👤 **Student Details** — подробна страница за студент
8. 😊 **По-добър интерфейс**
9. 🗄️ **Database** — текущата база вече е настроена и се използва
10. 🚪 **Login / Roles** — ако решим системата да има администратор и потребители

Основен архитектурен принцип

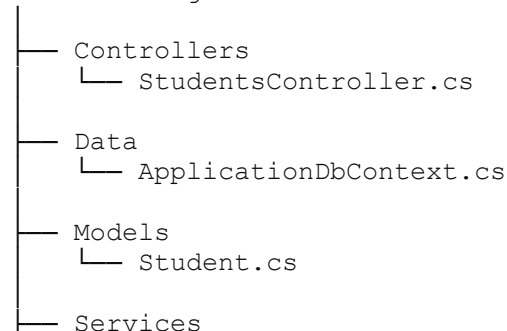
Изграждаме проекта **постепенно**, без предварително да създаваме ненужна сложност.

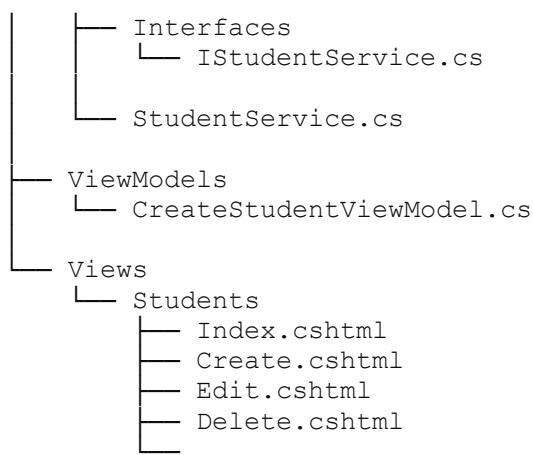
Всяко решение трябва:

- да работи добре за текущата функционалност;
- да позволява спокойно добавяне на следващите функционалности;
- да не налага ненужни patterns само „защото са patterns“.

2. Текуща структура на проекта

StudentManagement





Тази структура ще се развива постепенно.

Когато стигнем до Courses, Grades и Attendance, тогава ще добавяме съответните Controllers, Services, ViewModels и Models.

3. Основен план на проекта

#	Функционалност	Статус
1	 Students — CRUD	☒ В процес
2	 Search	☐ Предстои
3	 Courses / Subjects	☐ Предстои
4	 Grades	☐ Предстои
5	 Average Grade	☐ Предстои
6	 Attendance	☐ Предстои
7	 Student Details	☐ Предстои
8	 По-добър UI	☐ Предстои
9	 Database	☒ Настроена и използвана
10	 Login / Roles	☐ Предстои

Това е **общата пътна карта**, а не фиксиран списък.

Ако по време на разработката се появи по-добър подход, първо го обсъждаме и тогава променяме плана.

4. План на текущия рефакторинг

- [✓] Разглеждане на съществуващия проект
- [✓] Рефакториране към Service
- [✓] Създаване на Service интерфейс
- [✓] Отделяне на интерфейсите в Services/Interfaces
- [✓] Премахване на директния DbContext от Controller
- [✓] CRUD операциите преместени към StudentService
- [✓] Проверка с Build
- [✓] Създаване на ViewModels
- [✓] Създаден CreateStudentViewModel

- [✓] Create View използва CreateStudentViewModel
- [✓] POST Create приема CreateStudentViewModel
- [✓] Mapping ViewModel → Student

- [→] Проверка/рефакториране на Create.cshtml
- [] Edit ViewModel
- [] Edit.cshtml към Edit ViewModel
- [] Mapping при Edit
- [] Проверка/нормализация на whitespace
- [] Validation на входните данни
- [] Ограничения за дължини на полета
- [] Проверка на Age
- [] Security review
- [] Финален преглед на архитектурата
- [] Финално тестване на CRUD

5. Какво направихме досега

Първоначална архитектура

Първоначално Controller-ът работеше директно с Database Context:

```
StudentsController
    ↓
ApplicationDbContext
    ↓
EF Core
    ↓
Database
```

След рефакторирането

Вече имаме:

```
StudentsController
    ↓
IStudentService
    ↓
StudentService
    ↓
ApplicationDbContext
    ↓
EF Core
    ↓
Database
```

Целта е Controller-ът да не съдържа database/business logic.

6. StudentService

CRUD операциите вече са в StudentService:

```
GetAllAsync()
GetByIdAsync()
CreateAsync()
UpdateAsync()
DeleteAsync()
```

Controller-ът комуникира със Service чрез:

```
IStudentService
```

а реалната имплементация е:

```
StudentService
```

7. Какво разгледахме за EF Core

Разгледахме подробно:

- Add()
- SaveChangesAsync()
- FindAsync()
- Update()
- Remove()
- Change Tracking
- Unchanged
- Modified
- Deleted

Важно разграничение:

```
_context.Students.Add(student);
```

не означава, че студентът вече е записан в Database.

Реалното записване става при:

```
await _context.SaveChangesAsync();
```

Схемата е:

Student object

↓

DbSet.Add()

↓

Change Tracker

↓

SaveChangesAsync()

↓

EF Core

↓

SQL INSERT

↓

Database

8. CreateStudentViewModel

Създаден е:

ViewModels

└─ CreateStudentViewModel.cs

Той съдържа:

FirstName

LastName

Email

Age

Course

Validation:

FirstName

└─ [Required]

LastName

└─ [Required]

Email

└─ [Required]
└─ [EmailAddress]
└─ [StringLength(100)]

Age

└─ предстои допълнителна проверка

Course

└─ предстои да се прецени validation

Non-nullable warnings са оправени чрез:

= string.Empty;

9. Create flow

В момента потокът е:

Create.cshtml

↓

CreateStudentViewModel

↓

StudentsController

↓

Mapping

ViewModel → Student

↓

IStudentService

↓

StudentService

↓

ApplicationDbContext

↓

EF Core

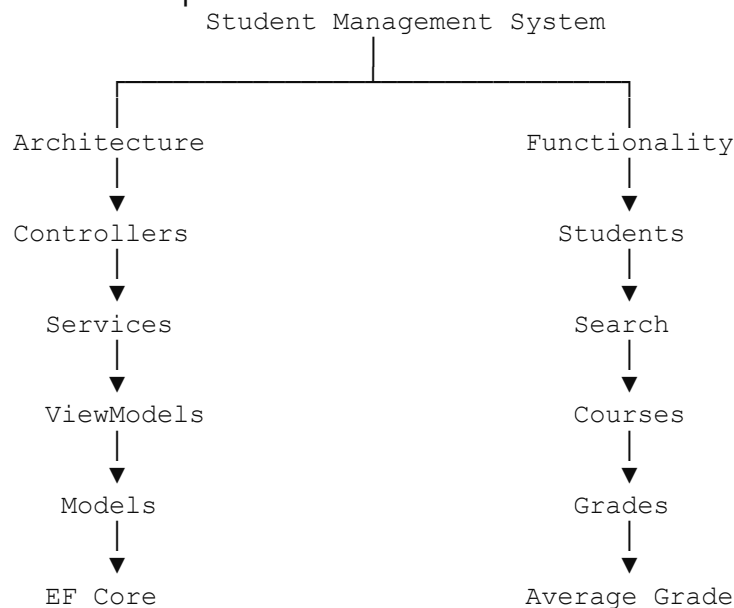
↓

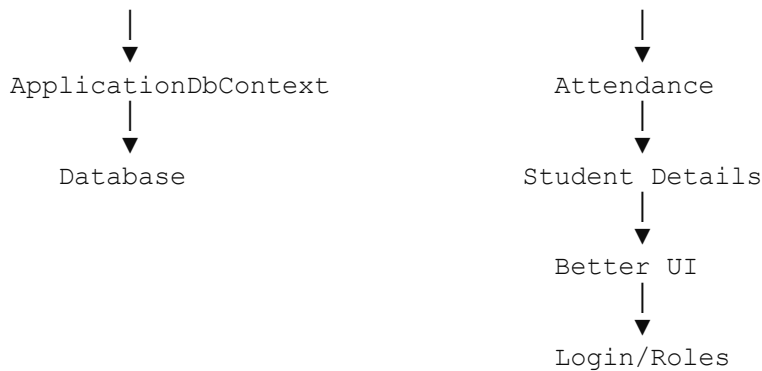
Database

Това е важната текуща архитектура.

10. Блок-схема на целия проект

Общата посока на проекта е:





11. Бъдеща архитектура

Когато проектът се разрасне, целта е постепенно да стигнем до структура от типа:

Controllers

- StudentsController
- CoursesController
- GradesController
- AttendanceController

Services

- Interfaces
 - IStudentService
 - ICourseService
 - IGradeService
 - IAttendanceService
- StudentService
- CourseService
- GradeService
- AttendanceService

ViewModels

- Students
- Courses
- Grades
- Attendance

Models

- Student
- Course
- Grade
- Attendance

Важно: Не създаваме всички тези файлове сега.

Добавяме ги, когато реално стигнем до съответната функционалност.

12. Решения, които вече сме взели

Service Pattern — ДА

Използваме:

```
Controller
  ↓
IStudentService
  ↓
StudentService
```

Controller-ът не трябва да съдържа database/business logic.

Интерфейси — отделна папка

Използваме:

```
Services
└─ Interfaces
```

Това е умишлено решение, за да бъде структурата ясна.

Repository Pattern — ЗАСЕГА НЕ

Обсъдихме Repository Pattern.

Решението е:

Не добавяме Repository само заради самия pattern.

Причината е, че проектът е сравнително малък, а EF Core вече предоставя data-access abstraction чрез:

```
DbContext
DbSet
```

Ако проектът порасне и се появи реална необходимост, Repository може да бъде добавен по-късно.

CSV / TXT Export

Ако по-късно поискаме:

```
Database
  ↓
CSV / TXT
```

не е задължително да използваме Repository.

По-логично е да имаме отделна export функционалност/service.

MySQL / друга Database

Ако по-късно сменим SQL Server с MySQL или друг provider, първо оценяваме реалната нужда от промяна в архитектурата.

EF Core поддържа различни database providers.

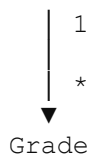
Repository не се добавя автоматично само защото сменяме Database.

13. Бъдещи връзки в Database

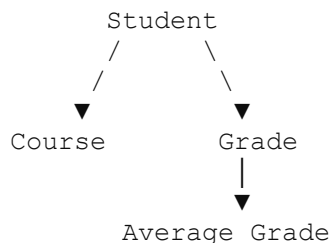
В момента Student не трябва да се разглежда като напълно изолирана таблица.

При Grades например очакваме връзка от типа:

```
Student
|
```



При Courses и Grades постепенно ще проектираме връзките между:



Точната Database структура ще бъде уточнена, когато стигнем до тези функционалности.

14. Как пишем проекта

Това е много важно.

Правило №1 — Стъпка по стъпка

Не правим 10 промени наведнъж.

Работим така:

1 малка промяна

↓
обяснение

↓
потребителят я прави

↓
Build

↓
проверка

↓
следваща стъпка

Правило №2 — Не гадаем кода

Ако трябва да променим съществуващ файл:

Потребителят изпраща файла

↓
Разглеждаме реалния код

↓
Обясняваме какво трябва да се промени

↓
Правим конкретната малка промяна

Особено важно за:

Controller

Service

View

ViewModel

DbContext

Правило №3 — Обяснение на начинаещо ниво

Не просто:

„Копирай този код.“

Обясняваме:

- какво прави кодът;
- защо го правим;
- кой слой отговаря за него;
- какво се случва с данните;
- как EF Core комуникира с Database.

Правило №4 — Обясненията са под кода

Например:

```
var student = await _studentService.GetByIdAsync(id);
```

След кода:

Service-ът търси студента по неговото ID.

Не натъпкваме излишни коментари вътре в кода.

Правило №5 — Използваме блок-схеми

При архитектурни и EF Core концепции използваме схеми.

Например:

Controller



Service



EF Core



DbContext



SQL Server



Database

При Create:

Create()



CreateAsync()



DbSet.Add()



Change Tracker



SaveChangesAsync()



SQL INSERT



Database

Правило №6 — Build след логична малка промяна

След всяка логична малка промяна проверяваме:

Build succeeded

Errors: 0

Warnings: 0

Едва тогава продължаваме.

Правило №7 — Security

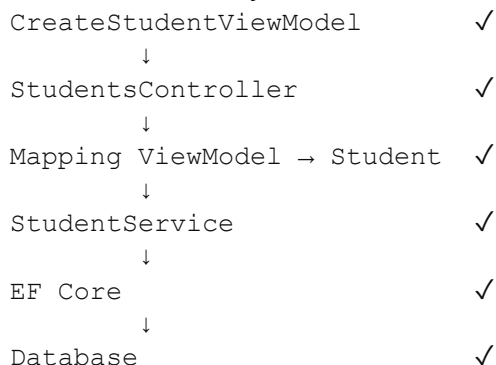
По време на разработката обръщаме внимание на:

- ✓ SQL Injection
- ✓ Validation
- ✓ Whitespace
- ✓ MaxLength
- ✓ XSS
- ✓ Mass Assignment
- ✓ CSRF
- ✓ Authorization
- ✓ Database constraints

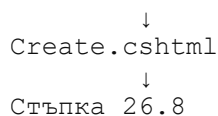
Вече обсъдихме, че нормалните LINQ заявки на EF Core са параметризирани и не трябва да сглобяваме SQL чрез string concatenation.

15. Точното място, от което продължаваме

В момента сме тук:



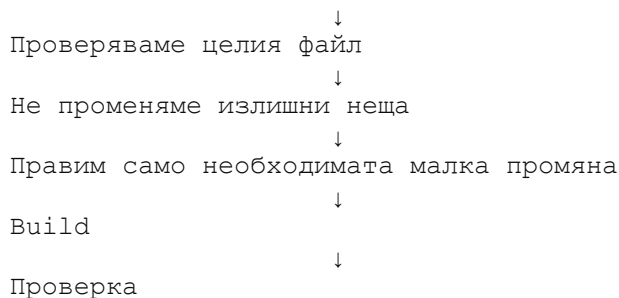
СЕГА:



CreateStudentViewModel вече е създаден.

Следващото действие е:

Потребителят изпраща актуалния Create.cshtml





16. Как продължаваме утре

Утре:

1. Давам GitHub линка към актуалния push.
2. Давам тази бележка.
3. Не започваме проекта отначало.
4. Първо възстановяваме текущото състояние.
5. Започваме директно от:

Стъпка 26.8



Create.cshtml

6. Потребителят изпраща реалния актуален код на Create.cshtml.
7. Разглеждаме го и правим само първата необходима промяна.

Ключово правило

Не започваме отначало.

Тази бележка + актуалният GitHub push са отправната точка за продължаване на проекта.

Целта е да продължим **точно от текущото състояние**, като запазим архитектурните решения и начина на работа, които вече сме договорили.

Това вече е форматът, който имаш предвид — **работна техническа бележка**, която утре можем да използваме като „карта“ на проекта.